

海量并发场景下的缓存架构设计

万俊峰Kevin

go-zero作者

好未来技术委员会资深专家



精彩继续！ 更多一线大厂前沿技术案例

📍 北京站

QCon

全球软件开发大会

时间：2021年5月29-31日

地点：北京 · 国际会议中心

扫码查看大会
详情>>



📍 北京站

GMITC

全球大前端技术大会

时间：2021年6月25-26日

地点：北京 · 国际会议中心

扫码查看大会
详情>>



📍 深圳站

ArchSummit

全球架构师峰会

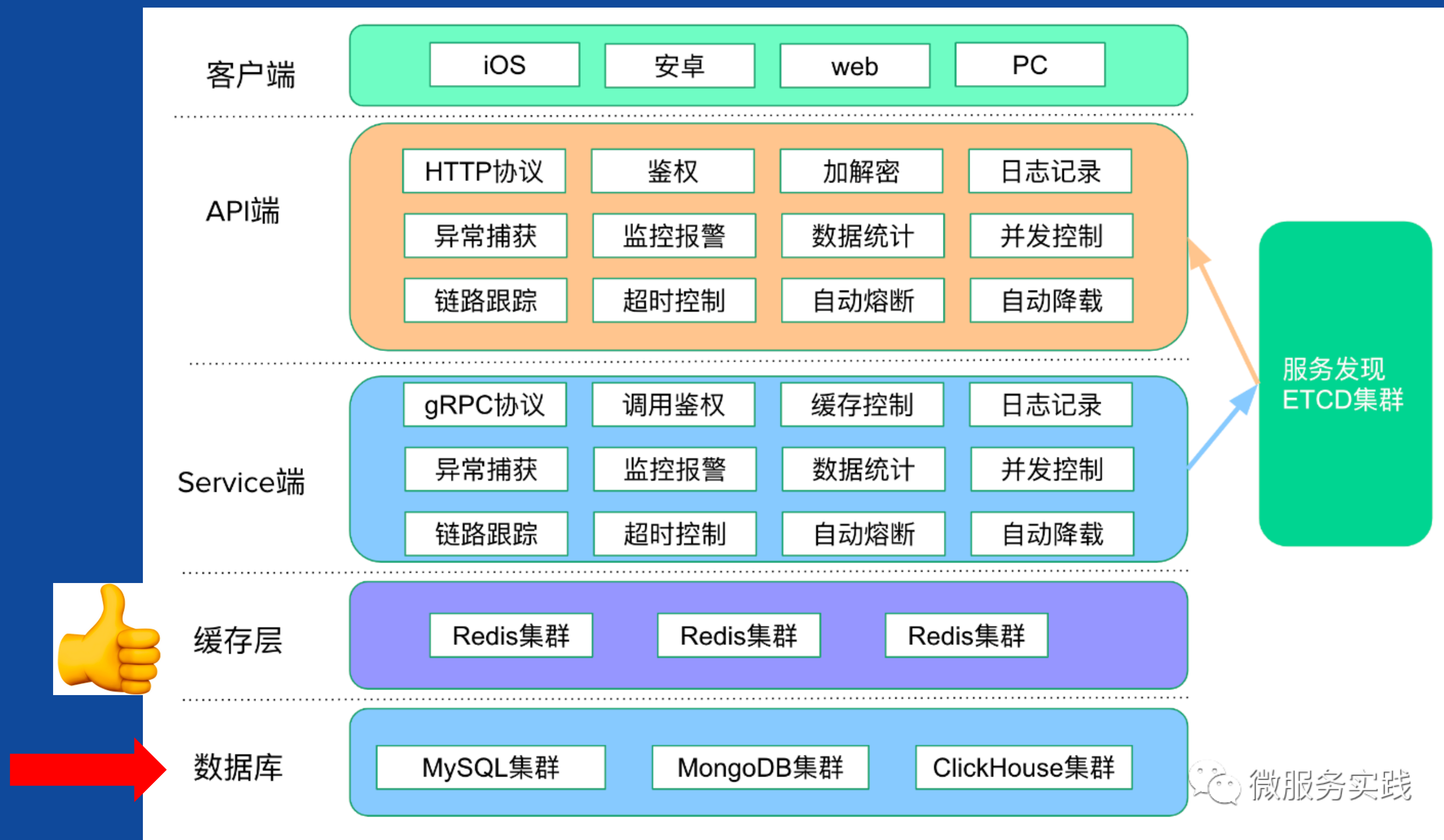
时间：2021年7月9-10日

地点：深圳 · 大中华喜来登酒店

扫码查看大会
详情>>



高并发下的系统瓶颈?

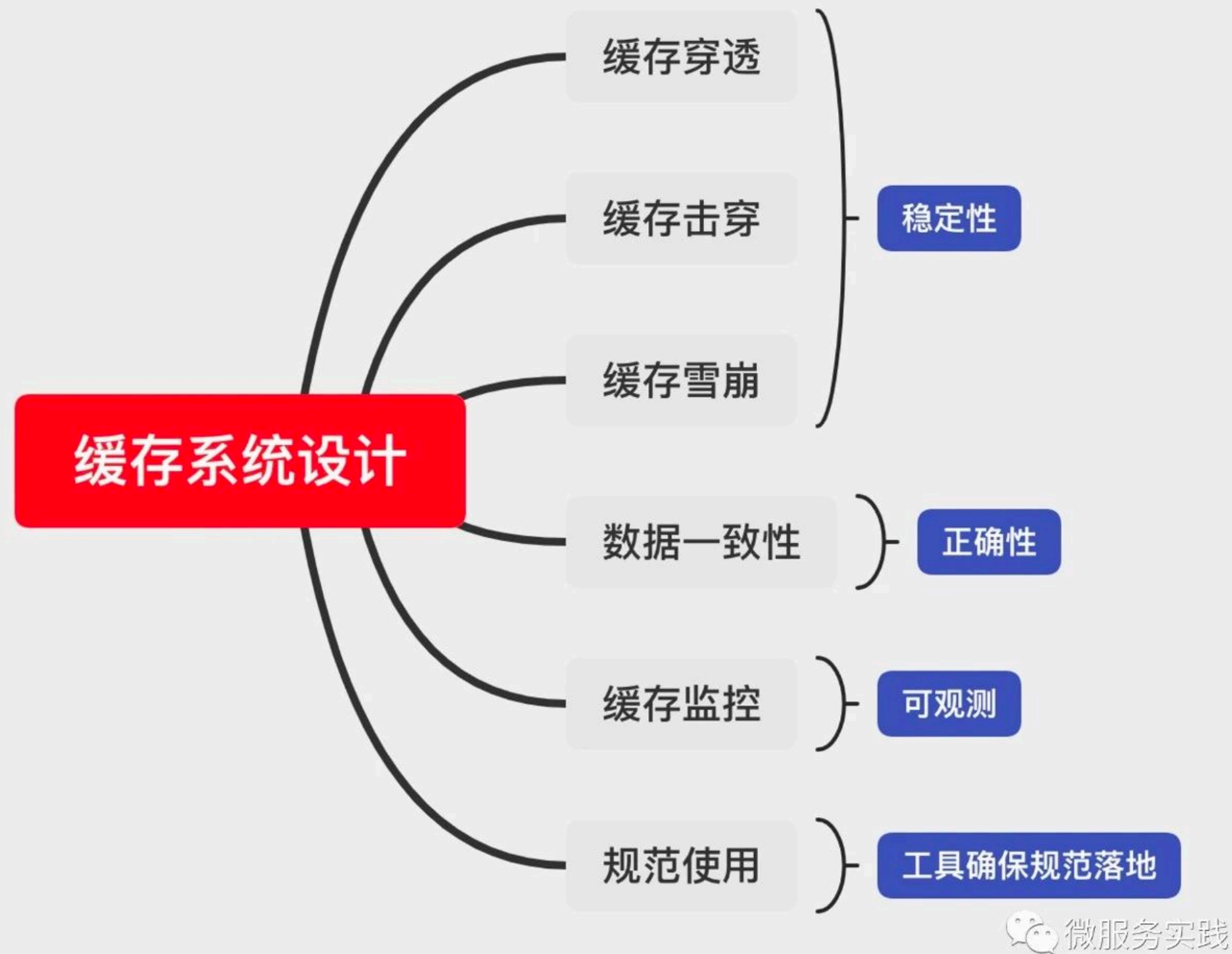


大纲

- 缓存系统常见问题
- 单行查询缓存与自动管理
- 多行查询缓存机制
- 分布式缓存系统设计
- 缓存代码自动化实践

缓存系统的复杂性和常见问题

缓存系统常见问题



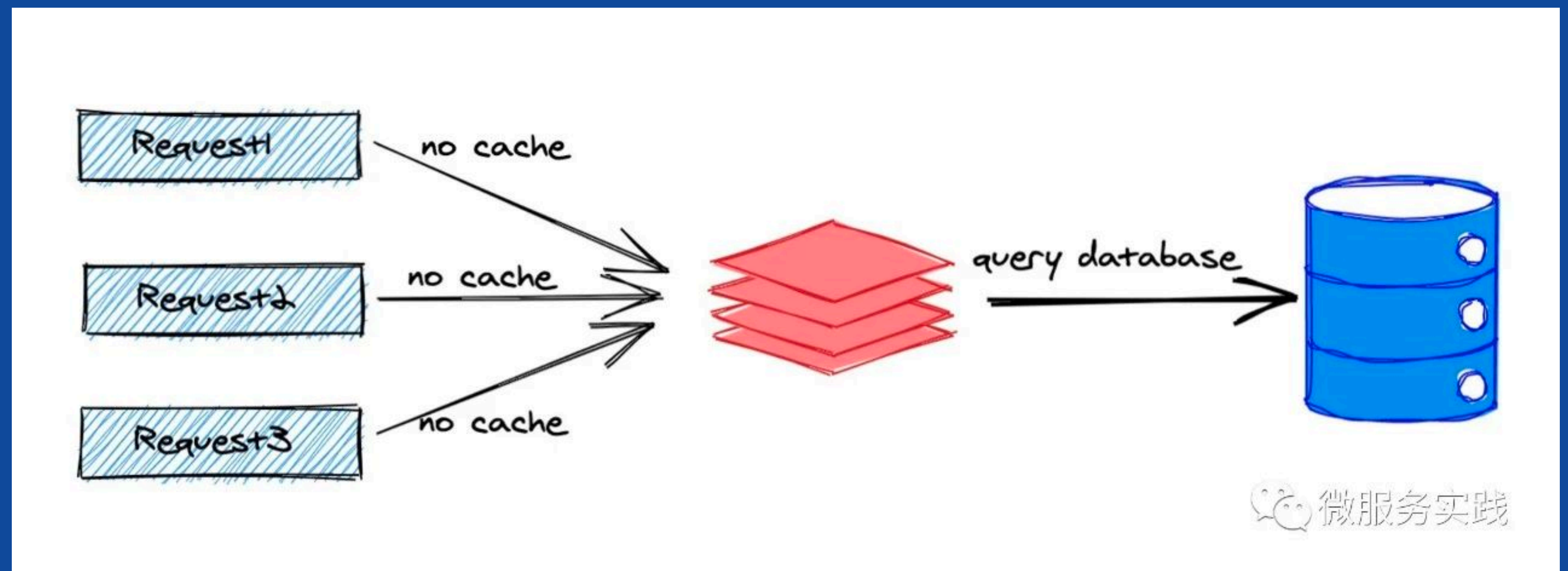
缓存穿透

问题原因：

- 不存在的数据请求
- 恶意请求

解决方法：

- 缓存占位符
- 短缓存，比如一分钟



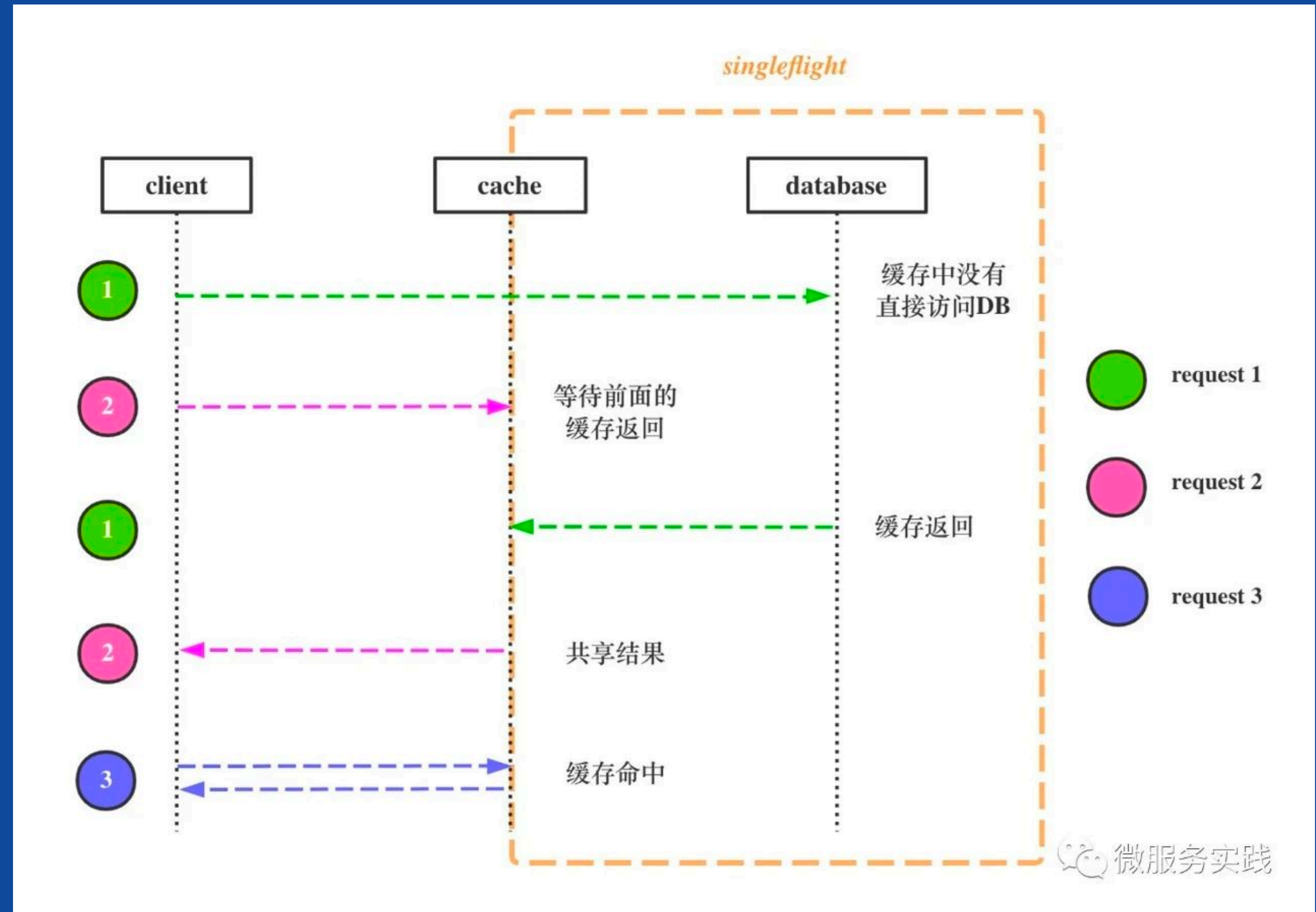
缓存击穿

问题原因：

- 热点数据过期
- 大量并发请求

解决方法：

- 唯一DB请求，共享结果
- 分布式锁



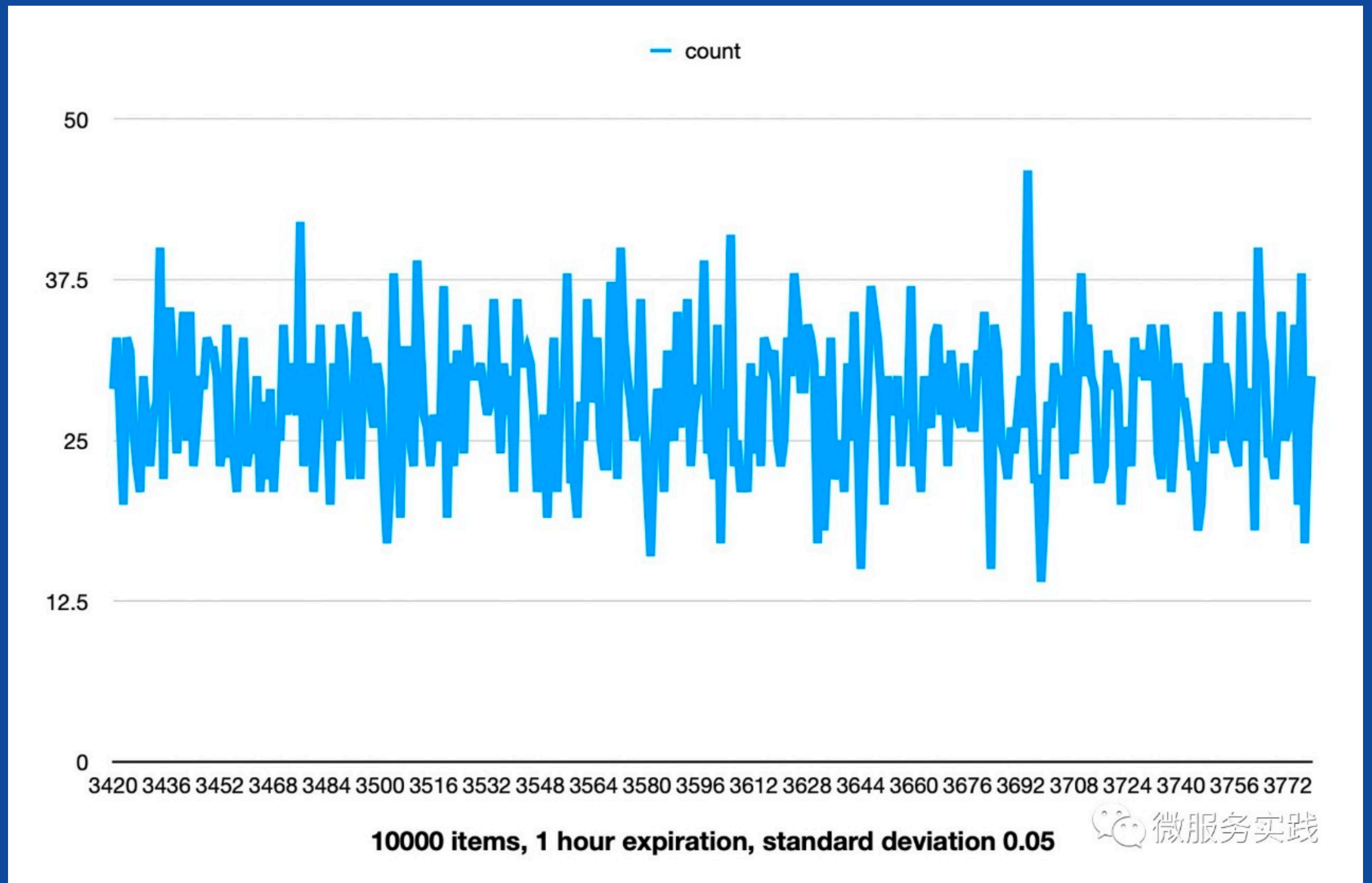
缓存雪崩

问题原因：

- 相同的缓存时间
- 缓存同时过期
- 大量请求落到DB

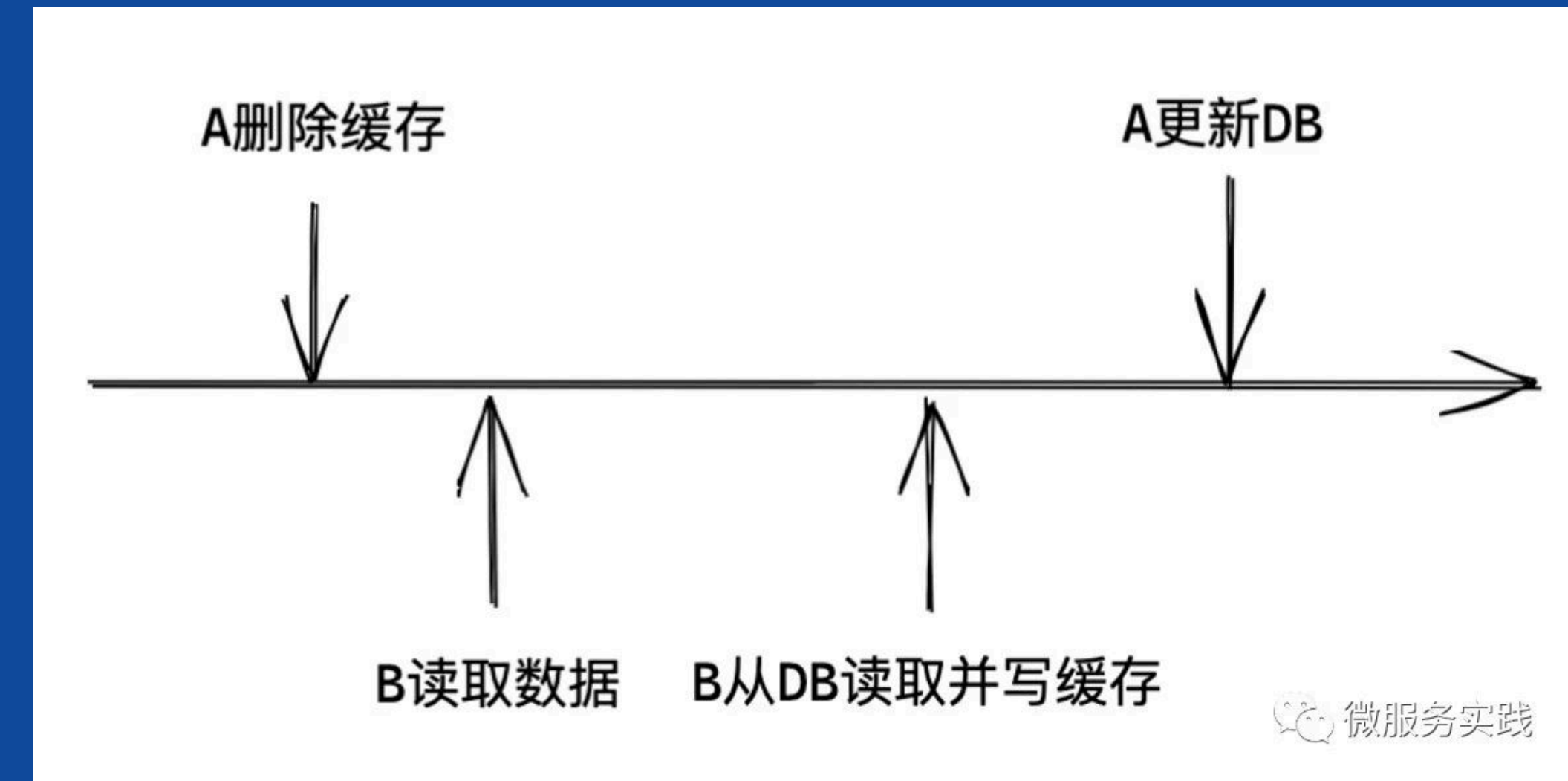
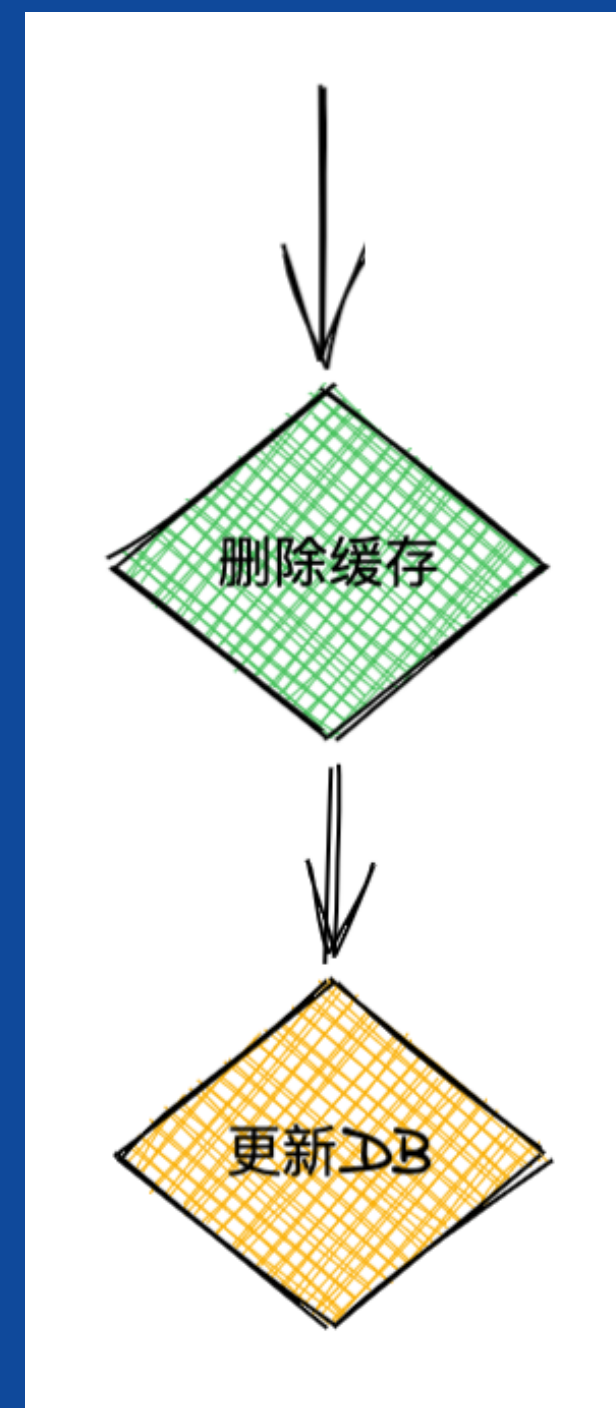
解决方法：

- 缓存过期时间加上随机偏差
- 分布式缓存，防止单节点故障



缓存一致性

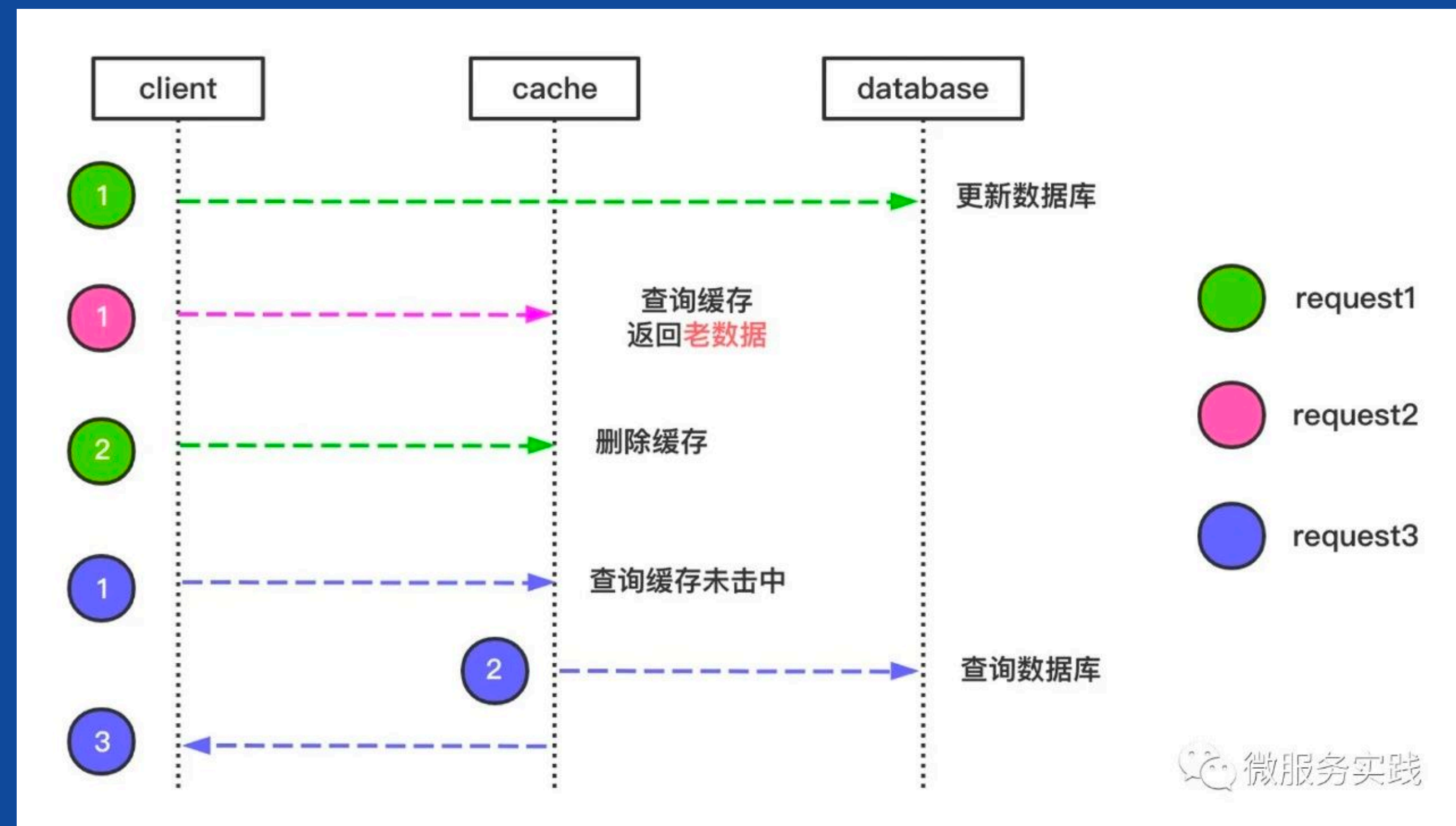
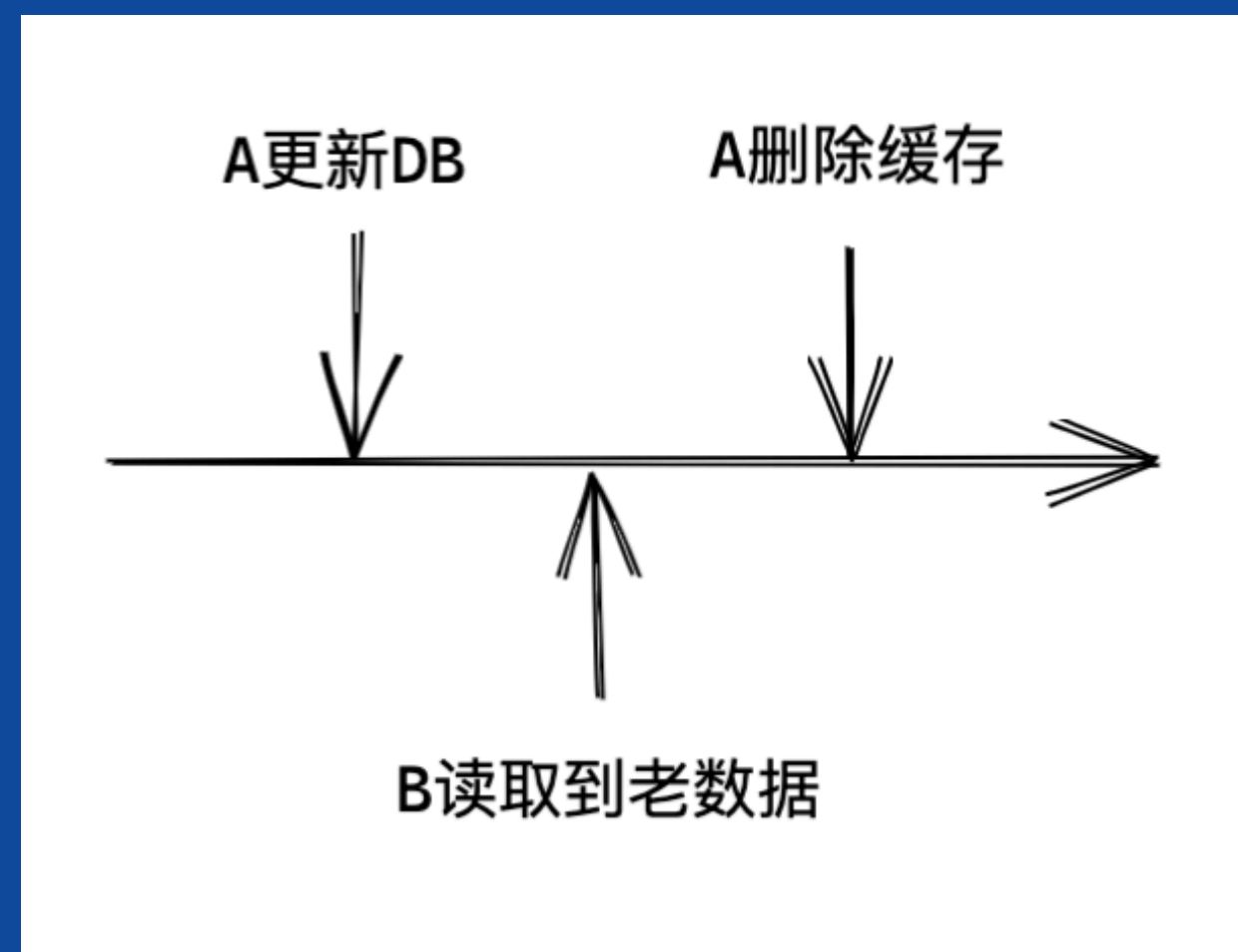
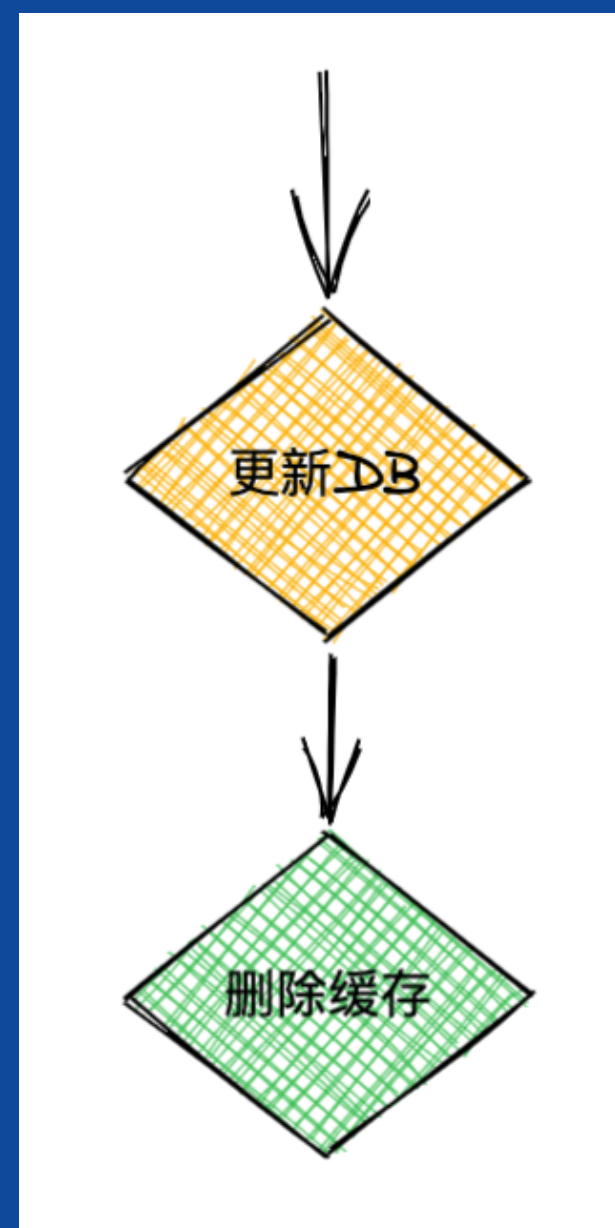
- 先删除缓存，再更新数据库



微服务实践

缓存一致性

- 先更新数据库，再删除缓存



缓存监控

- 缓存是否必需
- 过期时间是否合适
- 命中率如何
- 是否需要进一步调优

t	_index	🔍 🔍 📄 *	k8s_pro-2020.11.19
#	_score	🔍 🔍 📄 *	-
t	_type	🔍 🔍 📄 *	doc
t	content	🔍 🔍 📄 *	dbcache(sqlc) - qpm: 5057, hit_ratio: 99.7%, hit: 5044, miss: 13, db_fails: 0
?	k8s_cluster	🔍 🔍 📄 * ⚠️	pro4

微服务实践

如何让业务规范缓存的使用？

问题：

- 缓存知识点繁多
- 千人千面的缓存代码
- 很难写对

解决方法：

- 框架集成
- 工具一键生成

- 在 `rpc/model` 目录下执行如下命令生成CRUD+cache代码，`-c` 表示使用 `redis cache`

```
gocctl model mysql ddl -c -src book.sql -dir .
```

也可以用 `datasource` 命令代替 `ddl` 来指定数据库链接直接从schema生成

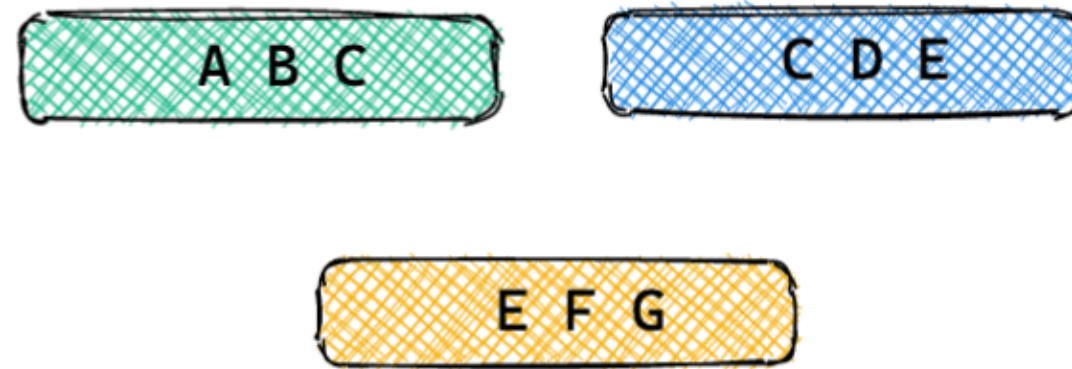
生成后的文件结构如下：

```
rpc/model
├── book.sql
├── bookstoremodel.go    // CRUD+cache代码
└── vars.go              // 定义常量和变量
```

微服务实践

单行查询的缓存与自动管理

单行查询的缓存



单行缓存分类

部分数据查询

不推荐使用，无法缓存（一致性难保障）

完整数据查询

基于主键查询

基于单列唯一索引查询

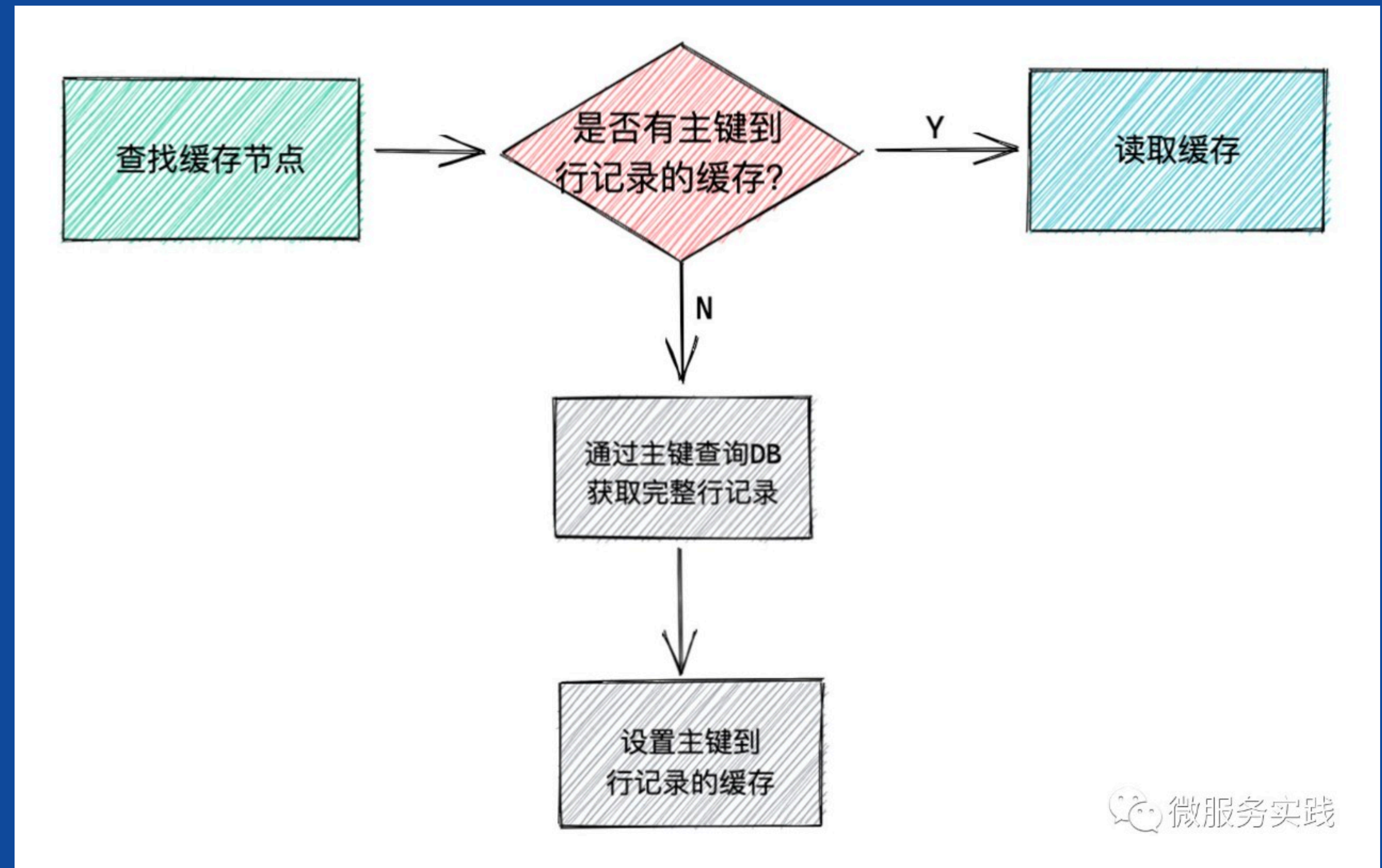
基于多列唯一索引查询

皆可缓存，自动生成

微服务实践

基于主键的单行缓存

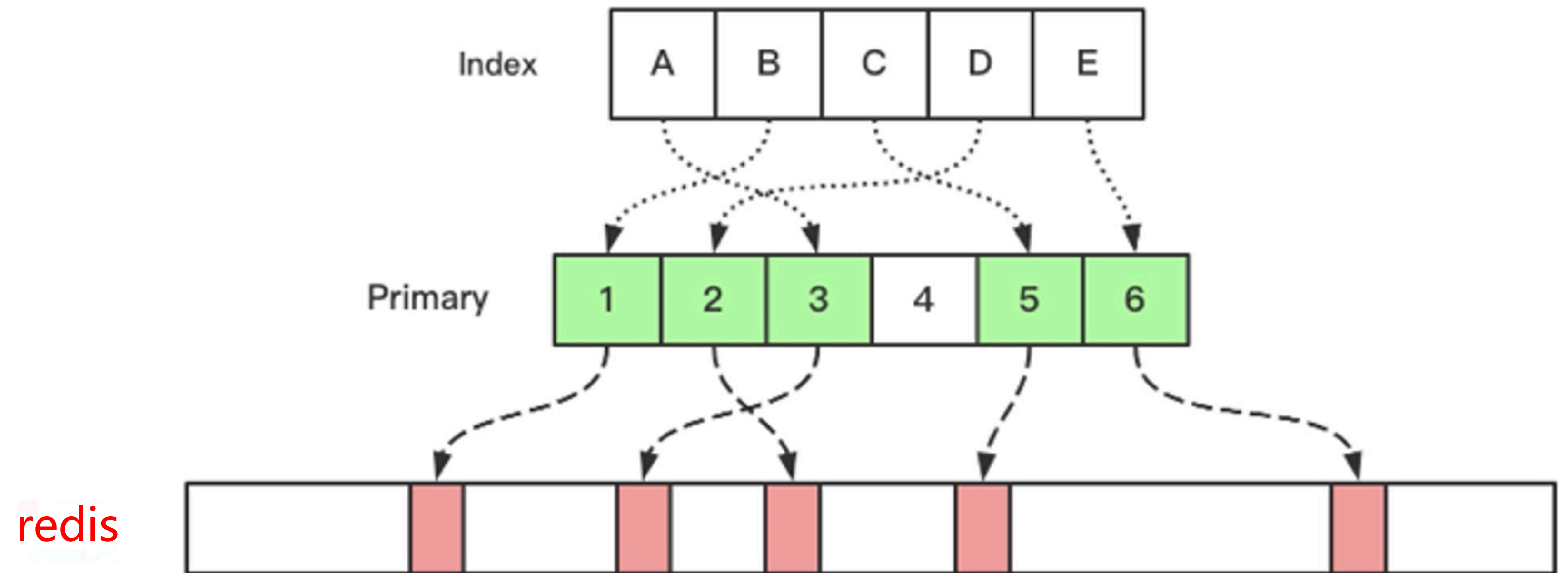
- 基于主键缓存
- 返回单行完整数据
- 自动生成代码



微服务实践

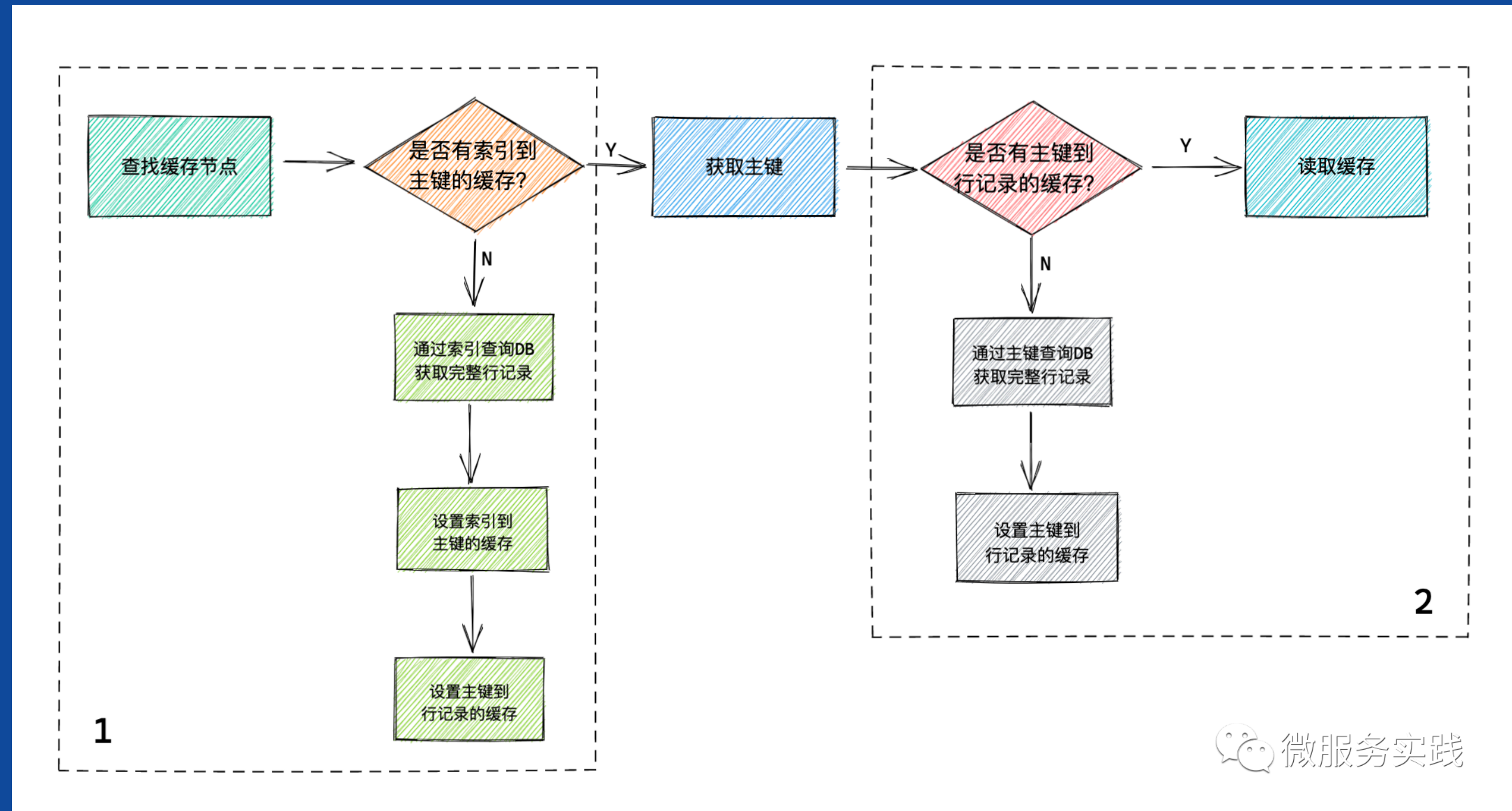
基于唯一索引的单行缓存设计

- 单列唯一索引
- 多列唯一索引
- 主键：完整数据



基于唯一索引的单行缓存逻辑

- 唯一索引：主键
- 主键：完整行数据
- 返回单行完整数据



单行缓存自动管理伪代码

- 自动防止数据不一致
- 自动防击穿
- 自动防穿透
- 自动防雪崩
- 自动统计

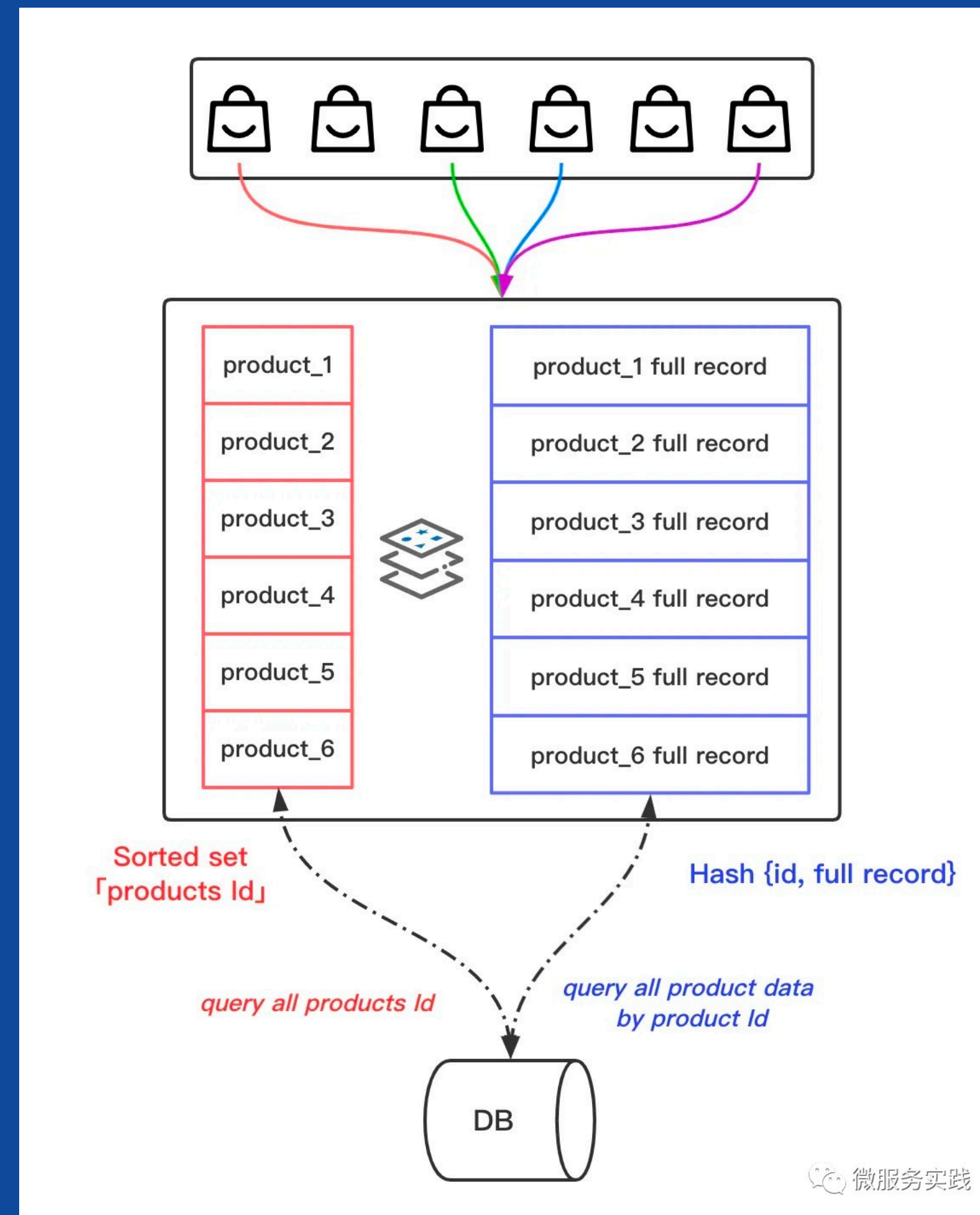
```
func QueryRow(...) error {  
    singleflight {  
        read cache  
        if placeholder {  
            return not found  
        }  
  
        if has cache {  
            return val  
        }  
  
        query db  
        if not found {  
            set cache with not found placeholder  
            return not found  
        }  
  
        set cache  
        return val  
    }  
}
```

完整实现代码见 [go-zero/core/stores](#) 下的 `cache`, `sqlc`, `mongoc` 子包

多行查询的缓存机制

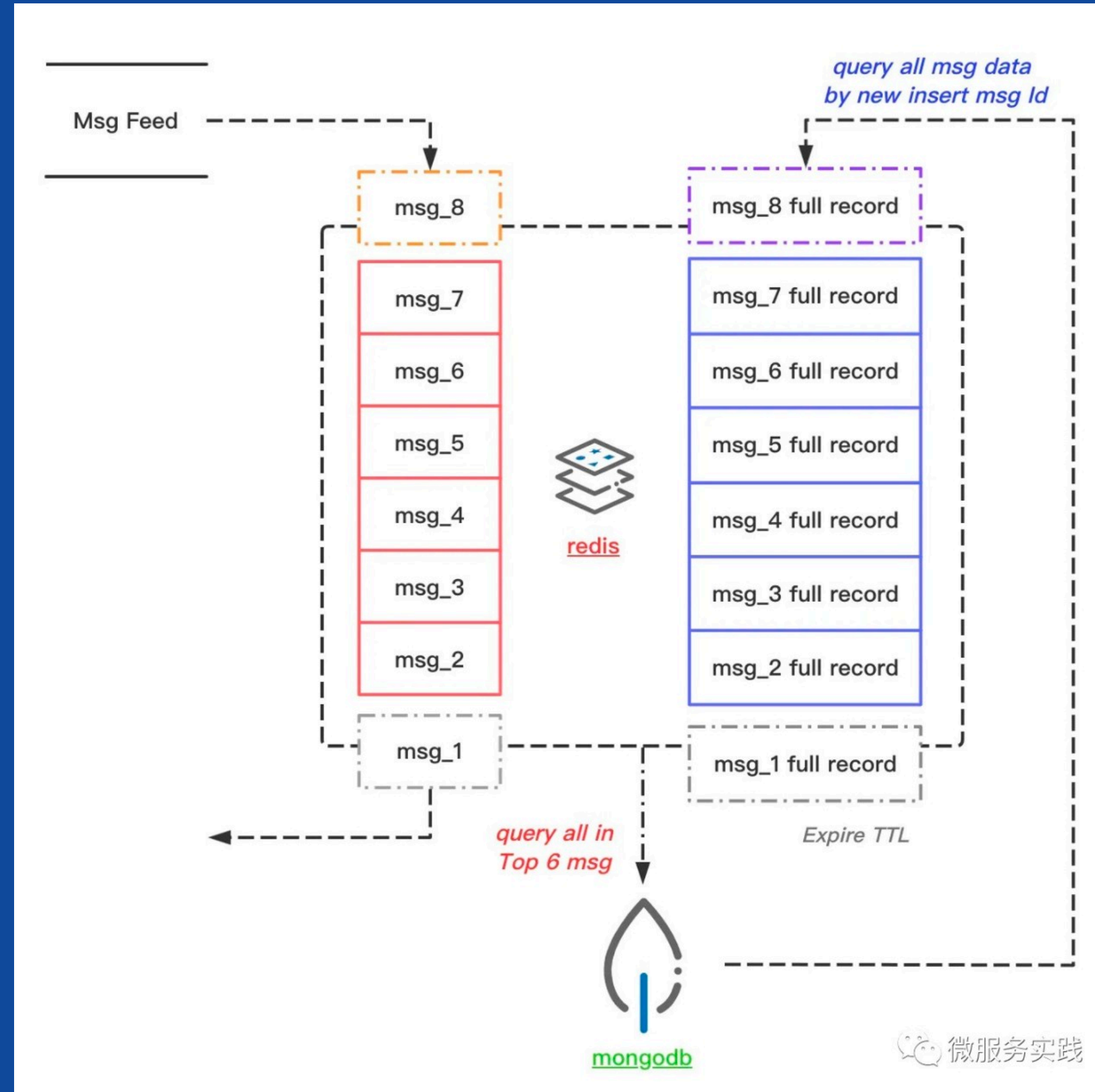
不易变的列表

- 比如店铺商品
- ID放在redis的sorted set
- 单列DB查询+缓存
- 修改友好，单条缓存删除
- mapreduce并发请求



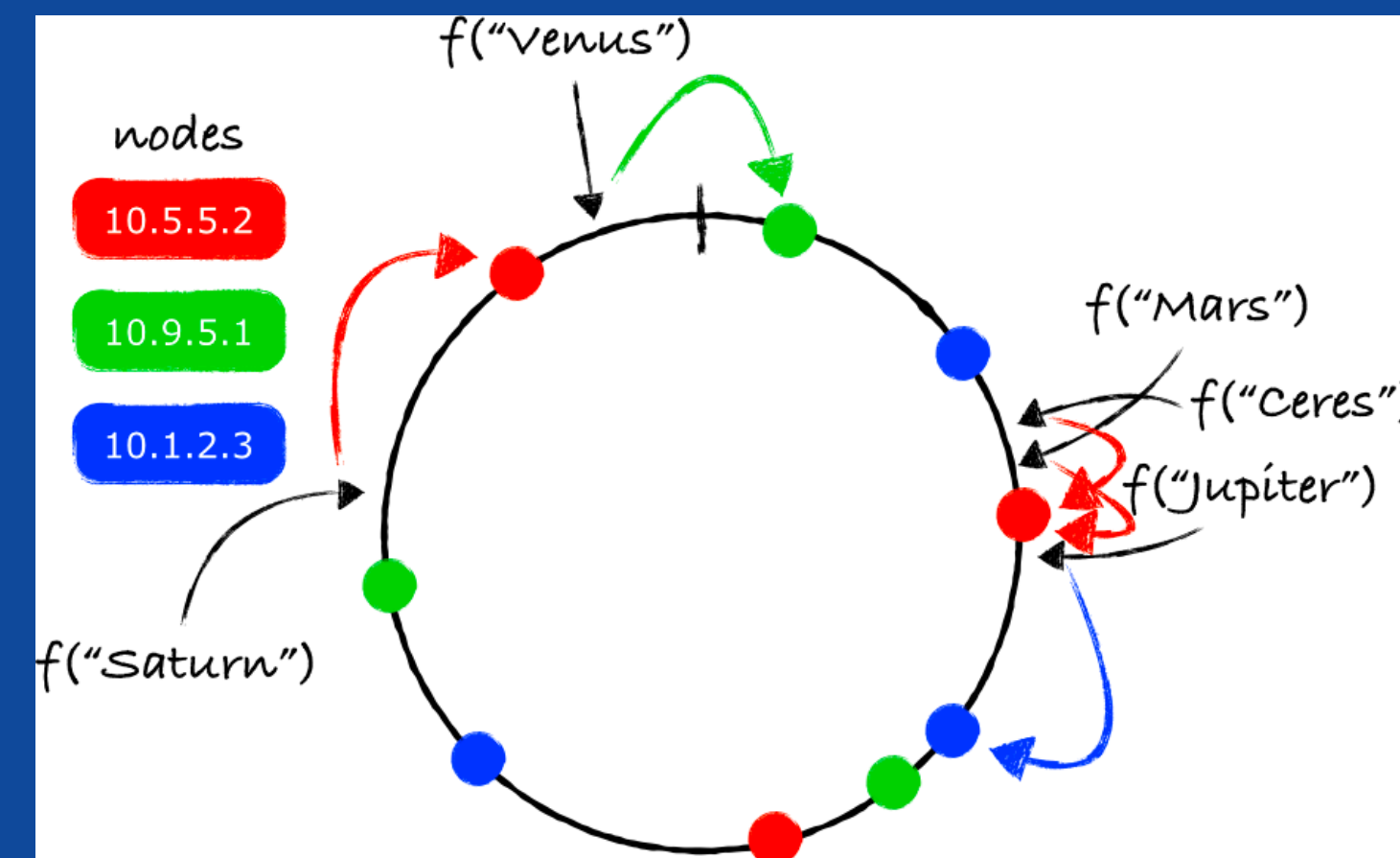
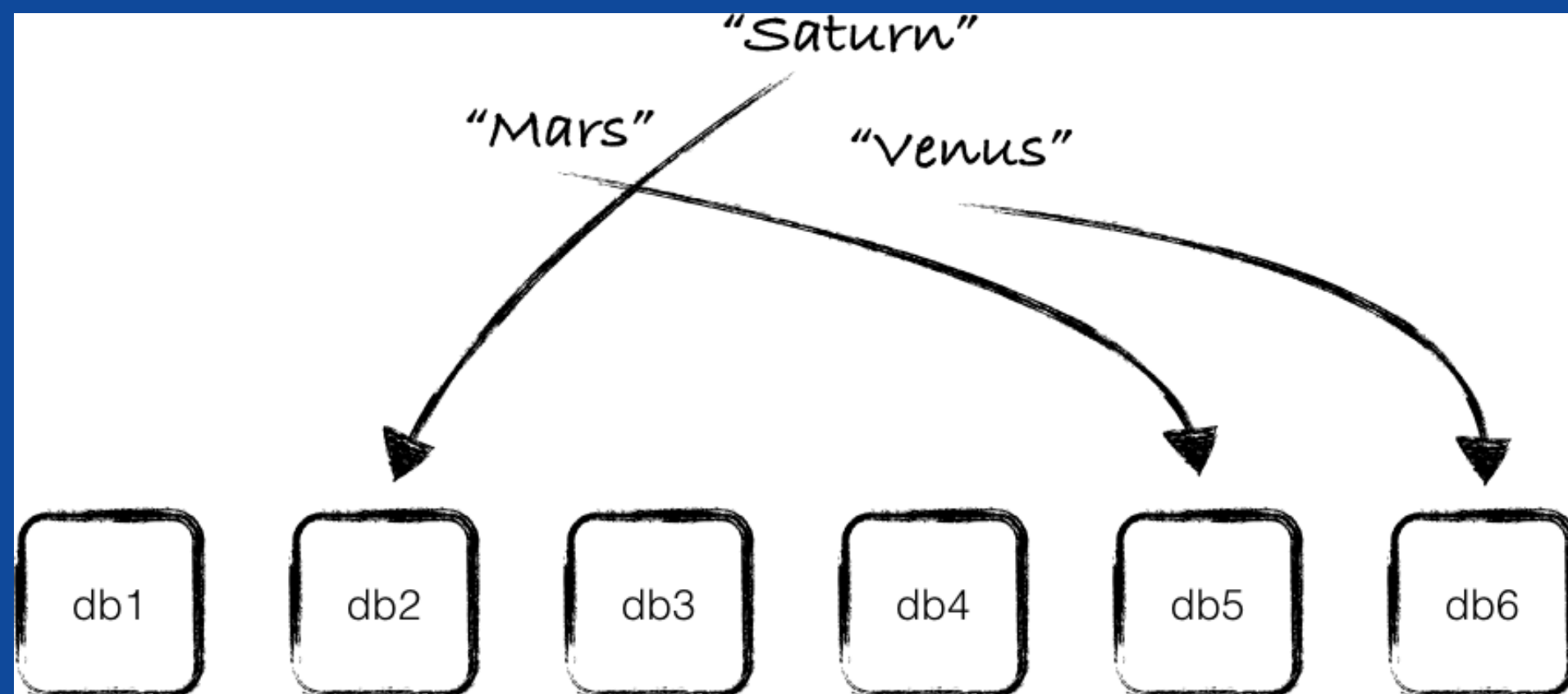
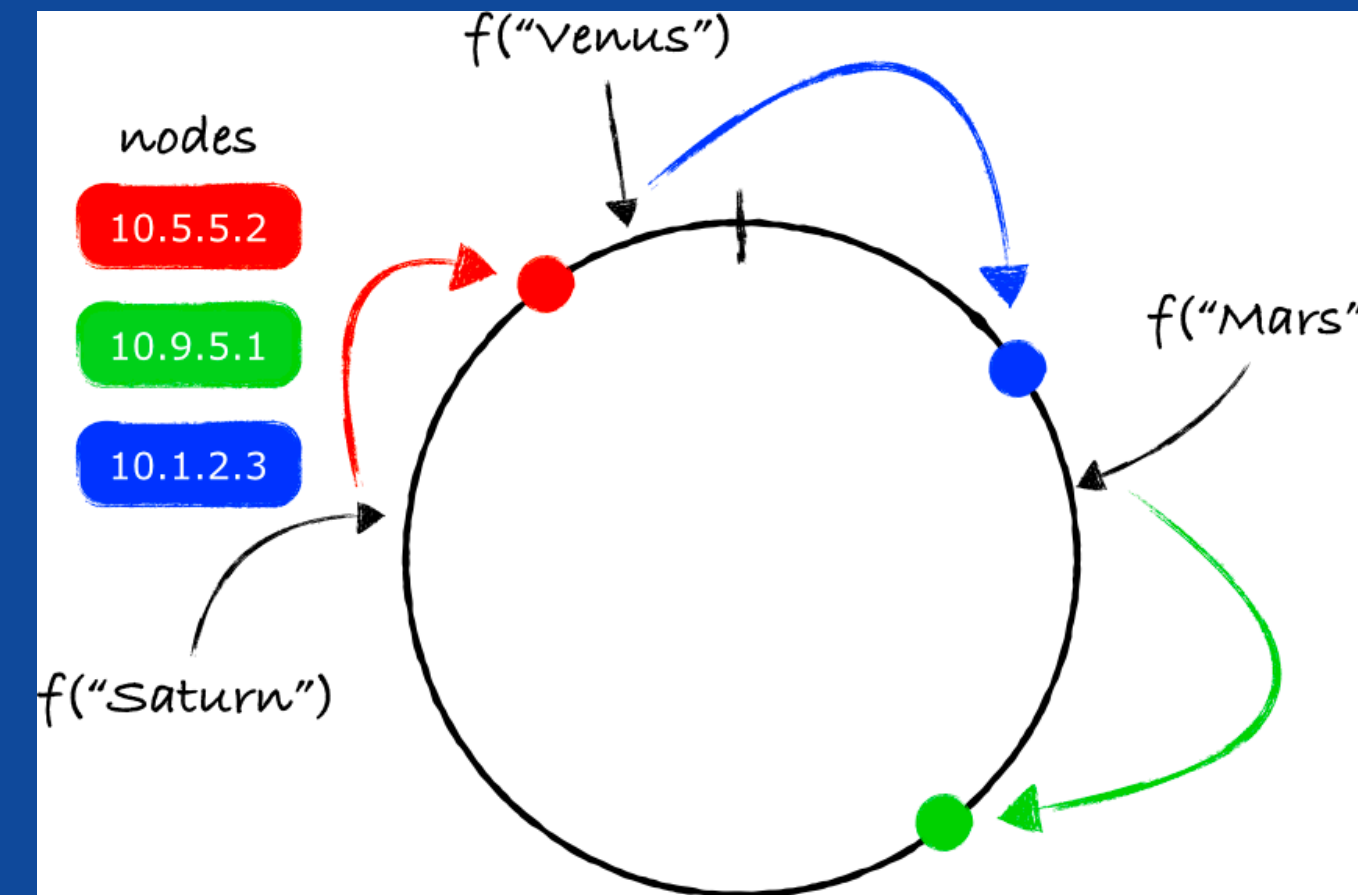
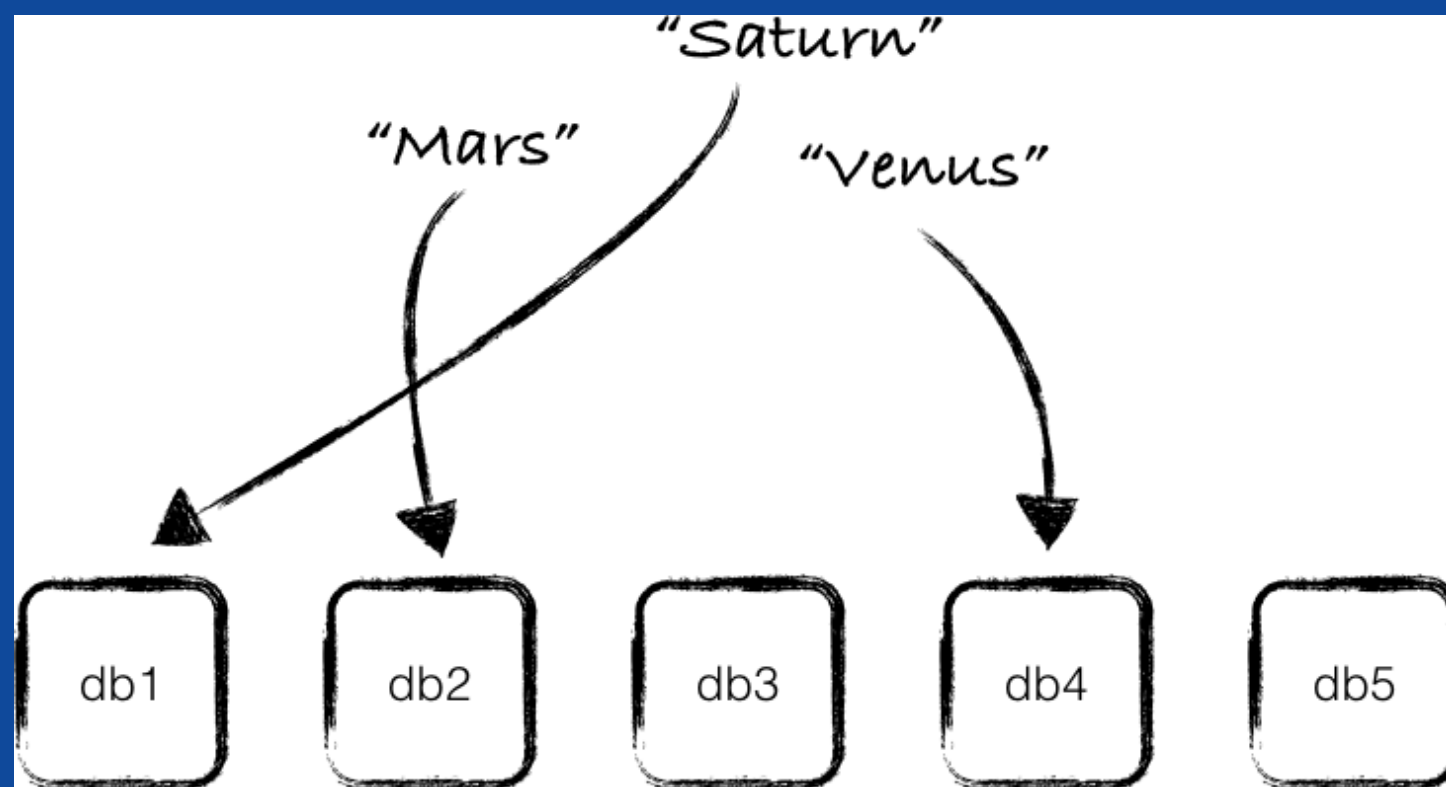
易变且分页的列表

- 比如信息流时间线
- ID放在redis的sorted set
- 前面加，后面减
- 单列DB查询+缓存
- mapreduce并发请求



分布式缓存系统设计

缓存分发算法



<https://blog.carlosgaldino.com/consistent-hashing.html>

缓存集群成本控制

- 数据量 vs QPS
- 数据量小、QPS高 => 多实例和分片
- Case study
 - 1TB redis cluster
 - 128GB redis clusters (16GB*8)

缓存代码自动化实践

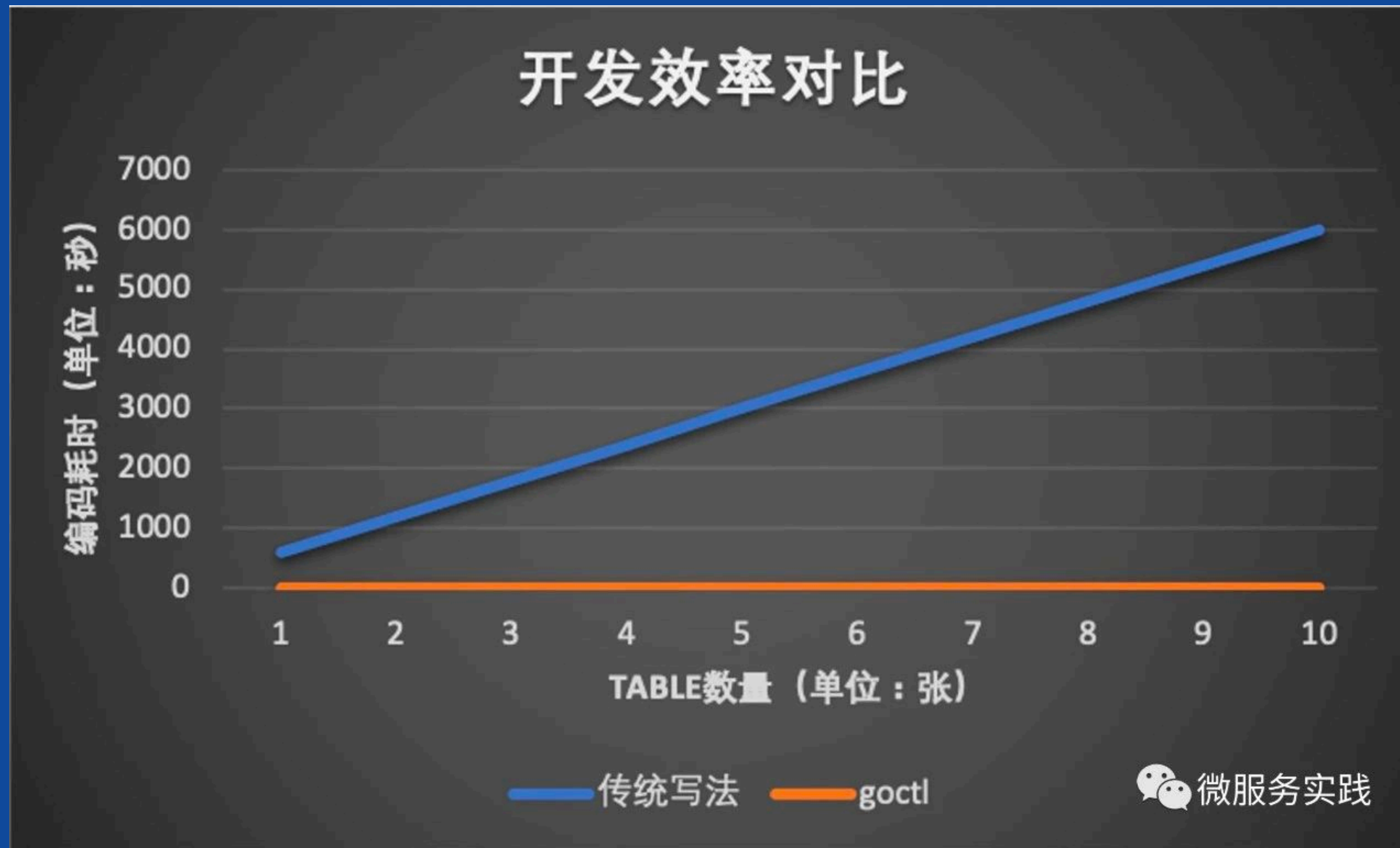
代码正确性 – 传统写法 vs goctl生成

- 怎么存放缓存数据
- 考虑缓存穿透、击穿、雪崩
- 如何保证数据一致性
- 如何监控缓存使用情况
- 手撸代码越多，bug越多
- sqlc, mongoc自动管理缓存
- Nothing else!

开发效率 – 传统写法 vs goctl生成

- 10分钟一个表的CRUD
- 30分钟一个表的缓存代码
- table -> 结构体
- code review, test
- ddl (sql文件) 一键生成
- 同一DB下多表一次生成
- 自动生成sql, 自动检测问题

开发效率 – 传统写法 vs goctl生成



懂原理，用工具

—

Global

Architect Summit

QCon⁺ 案例研习社



扫码学习大厂案例

学习前沿案例，向行业领先迈进

40个

热门专题

—
行业专家把关内容筹备，
助你快速掌握最新技术发展趋势

200个

实战案例

—
了解大厂前沿实战案例，
为 200 个真问题找到最优解

40场

直播答疑

—
40 位技术大咖，每周分享最新
技术认知，互动答疑

365天

持续学习

—
视频结合配套 PPT
畅学 365 天